Bluetooth-Operated Mini Tank: Autonomous Modes and LED Matrix Interface

Amman Mirza

Abstract

This project provides an overview of the development of a holistically remote-controlled tank robot that can run autonomously as well as under manual control. It utilizes Bluetooth radio signals in communication with an Android phone for manual control in real-time. It also features multiple automated modes including obstacle detection, light tracking, and target tracking, all programmed with the help of ultrasonic distance sensors and photoresistors. Obstacle detection enables the robot to detect objects ahead and manoeuvre through the obstacles in a secure manner, adding reliability and flexibility in the unknown terrain. Light tracking mode enables the robot to follow a light source with varying input of two photoresistors, demonstrating a straightforward example of interaction with the environment. Follow mode regulates at a specific distance from an object by continuously measuring distance with the use of the ultrasonic sensor. A visual feedback system is implemented using an LED matrix display to notify the user of the current mode and status, improving usability and interactivity. The design addresses problems related to the merging of wireless communication with sensor data processing and motor control within a constrained hardware environment. The project demonstrates how the merging of manual control with autonomous features can lead to an interactive and responsive robotic platform. This project lays the groundwork for further development in intelligent system design and embedded robotics.

Introduction

Robotics today pervades numerous disparate fields, ranging from industrial automation to household work. Contributing in part to the wide-ranging application is the convergence of the application of environmental sensing, embedded systems, and wireless networking. The project here entails the design and implementation of an autonomous, remotely controlled robot for surveillance, exploration, and navigation purposes. With the symbiotic usage of environmental sensing and remote Bluetooth control, the robot is able to function in both manual and autonomous modes, being able to adjust its behaviour in response to its environment.

The use of Bluetooth technology facilitates manual, real-time control through an Android smartphone. The robot is controllable remotely through commands, delivering low-latency control and uniform response towards user commands. Wireless control enhances the viability through remote robot control, delivering convenience and enhancing its practicability in real-time operations.

Autonomy is achieved through the use of multiple sensors. Obstacle detection and avoidance are achieved through an ultrasonic distance sensor on a servo motor. The robot can detect distances in the front and sides and makes intelligent decisions in regard to rerouting and avoiding collision. Obstacle avoidance is an elementary idea in mobile robotics and is extremely critical while navigating through an unstructured setting.

Light-tracking motion is created through two photoresistors that sense intensity in ambient light. The robot turns in the direction towards the brighter light, providing an instance of simple environmental-reactive motion. In addition, an object-follow mode incorporates the ultrasonic sensor in order to trail an object or person at a desired, safe distance, bringing an interactive factor to the system.

A 16x8 display screen offers real-time visual response, displaying the active mode and the direction commands. It improves user experience, facilitates the process of debugging, and enhances the operational transparency.

The project demonstrates the feasibility for constructing an affordable, modular robot that harnesses sensor fusion, wireless connectivity, and onboard computation for intelligent action. It reflects the directions for subsequent advancement in autonomous systems, real-time control in embedded systems, and autonomous robotics.

Materials and Methods

The robot tank was constructed using the microcontroller Arduino Uno R3 and could work in manual and auto modes. It has Bluetooth-based remote-control characteristics combined with smart features including obstacle detection, following an object, and tracking light. The entire system was constructed from the electronic component parts comprising some educational kits and were integrated into an in-house-constructed body.

Microcontroller and Expansion

The core of the system lies in the use of the Arduino Uno R3, which handles sensor input, motor output, and the control of wirelessly attached devices. To simplify the connection of multiple components, the sensor shield was placed on top of the Arduino. Such an approach made the wiring simpler through the grouping into power, ground, digital I/O, and PWM signals, ensuring the build was an efficient and tidy one.

Locomotion System

The movement is controlled with two DC motors, whose operations are coordinated through the motor drive module. The motors are driven by the PWM signals from the Arduino for the varying speeds, and the digital outputs control the directions. The differential drive mechanism gives the forward, backward, left, and right movement and the in-place turns. The PWM adjustment in the speed avoids the robot losing its stability while in autonomous movement.

Wireless Bluetooth Control

It can be operated manually through an Android app. It provides serial commands to the Arduino character by character. It gives commands like an 'F' for forward, 'B' for backward, or an 'X' for autonomous light tracking for their corresponding activities. The 'Z' stop command ends any mode and the tank halts. It is an interference-free, heavy-duty system.

Sensor Integration

The tank relies on numerous sensors to perform autonomous operations:

- Ultrasonic Distance Sensor on Servo Motor: Located in the front, the ultrasonic sensor operates on the principle of the reflection of sound waves to calculate distance. Housed in the servo, the sensor can sweep left and right, giving it an enhanced visibility. Obstacle detection and tracking moving objects rely on the mechanism for sweeping.
- Light-dependent resistors (LDRs): Two analog light-dependent resistors are mounted on the front corners of the tank. They track the ambient light intensity and are used for moving the tank toward brighter areas. The tank turns and goes in the direction of the light source based on the comparison between the intensity of the light on both sides.
- IR Receiver: IR receiver is connected at the digital input pin for the potential for future support for remote control. It is not currently implemented in the implementation; however, it does provide the system expandability.

Analog and digital pins are used for acquiring the sensor inputs, and the special code in the Arduino is specifically designed for reading the input and manipulating the movements of the tank accordingly.

Power Supply

There is power supplied through two 3.7V 3300mAh batteries, sufficient for both the motors and the electronics. They power the sensor shield power rails, supplying the modules. Decoupling capacitors were used for voltage spiking filtering, mostly while accelerating the motors very rapidly and when the direction changes too.

LED Matrix Display

The 16x8 LED matrix offers graphical feedback in real-time. I2C-powered, it displays icons for the state of the tank now, including direction arrows and distinctive patterns for every autonomous state. Bitmap icons are stored in advance in arrays and modified in the real-time Arduino loop for function changes, maximizing user interactivity.

Autonomous Modes

The robot possesses three primitive autonomous behaviours:

1. Obstacle Avoidance: The tank stops whenever it finds an object at 20 cm. The ultrasonic sensor is swung left and right by the servo, reading both directions. The tank turns in the direction it finds clear space and goes forward.

2. Object Following: The vehicle moves forward towards the object when it is in 10–20 cm proximity. It stops or adjusts according to the need while keeping the proper following distance if the object is too distant or too near.

3. Light tracking: The two photoresistors detect the amount of light. When it is brighter in one direction, the tank turns in the brighter direction. When both photoresistors encounter bright light, the tank goes forward. When the two photoresistors both experience shade, the tank stops.

All the modes are activated through Bluetooth and run continuously until interrupted through the stop function.

Software Logic and Control

The Arduino code utilizes the mode-based switch design, responding to the Bluetooth inputs and calling the relating routines. The motor control routine, Tank_forward() or Tank_turnLeft(), is encapsulated and reused in behaviors. The sensor inputs are smoothed through averaging, specifically for ultrasonic inputs. Delays in servo moves retain scan stability. The robot stays in autonomous loops through the flag mechanism until the stop command, allowing for smooth switch-off between manual and autonomous control.

Code Implementation:

RC_Tank §

```
unsigned char start01[] = {0x00, 0x7c, 0x12, 0x12, 0x7c, 0x00, 0x7e, 0x02, 0x7e, 0x02, 0x7e, 0x00, 0x44, 0x7e, 0x40, 0x00};
unsigned char front[] = {0x00, 0x00, 0x80, 0xf0, 0xf8, 0x78, 0x2e, 0xf3, 0xf3, 0x2e, 0x78, 0xf8, 0xf0, 0x80, 0x00, 0x00};
unsigned char back[] = {0x00, 0x00, 0x01, 0x1a, 0x0e, 0x36, 0x62, 0x82, 0x82, 0x62, 0x36, 0x0e, 0x1a, 0x01, 0x00, 0x00};
unsigned char left[] = {0x00, 0x00, 0x00, 0x18, 0x3c, 0x7e, 0xe7, 0xc3, 0x81, 0x00, 0x18, 0x18, 0x18, 0x7e, 0xdb, 0x99};
unsigned char right[] = {0x99, 0xdb, 0x7e, 0x18, 0x18, 0x18, 0x00, 0x81, 0xc3, 0xe7, 0x7e, 0x3c, 0x18, 0x00, 0x00, 0x00};
unsigned char STOP01[] = {0x7e, 0x08, 0x7e, 0x00, 0x7e, 0x0a, 0x7e, 0x00, 0x7e, 0x40, 0x40, 0x02, 0x02, 0x7e, 0x02, 0x02};
unsigned char followmode[] = {0x00, 0x00, 0xf8, 0x48, 0x50, 0xf0, 0x14, 0x04, 0x04, 0x1f, 0x0e, 0x04, 0x60, 0x7c, 0x7c, 0x7c};
unsigned char light[] = {0x00, 0x08, 0x49, 0x2a, 0x00, 0x00, 0x3c, 0x7e, 0x7e, 0x3c, 0x00, 0x00, 0x2a, 0x49, 0x08, 0x00};
unsigned char obstAvoid[] = {0x00, 0x00, 0xf8, 0x48, 0x50, 0xf0, 0x10, 0x00, 0x28, 0x44, 0x00, 0xfc, 0xfc, 0xfc, 0xfc, 0xfc};

unsigned char clear[] = {0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00};

#define SCL_Pin  A5
#define SDA_Pin  A4
#define ML_Ctrl 13
#define ML_PWM 11
#define MR_Ctrl 12
#define MR_PWM 3
#define Trig 5
#define Echo 4
#define servoPin 9
#define light_L_Pin A1
#define light_R_Pin A2

int distance, a, a1, a2, random2, left_light, right_light;
char bluetooth_val;
int flag;

unsigned long lastStopTime = 0;
bool isStopped = false;
bool isAutoMode = false;

void setup() {
  Serial.begin(9600);
  pinMode(Trig, OUTPUT);
  pinMode(Echo, INPUT);
  pinMode(ML_Ctrl, OUTPUT);
  pinMode(ML_PWM, OUTPUT);
  pinMode(MR_Ctrl, OUTPUT);
  pinMode(MR_PWM, OUTPUT);
  pinMode(servoPin, OUTPUT);
  pinMode(SCL_Pin, OUTPUT);
  pinMode(SDA_Pin, OUTPUT);
  pinMode(light_L_Pin, INPUT);
  pinMode(light_R_Pin, INPUT);
  procedure(90);
  matrix_display(clear);
  matrix_display(start01);
}

void loop() {
  if (Serial.available()) {
    bluetooth_val = Serial.read();
    Serial.println(bluetooth_val);
  }
```

RC_Tank

```
void loop() {
  if (Serial.available()) {
    bluetooth_val = Serial.read();
    Serial.println(bluetooth_val);
  }

  switch (bluetooth_val) {
    case 'F': isAutoMode = false; Tank_front(); matrix_display(front); break;
    case 'B': isAutoMode = false; Tank_back();  matrix_display(back); break;
    case 'L': isAutoMode = false; Tank_left();  matrix_display(left); break;
    case 'R': isAutoMode = false; Tank_right(); matrix_display(right); break;
    case 'S': isAutoMode = false; Tank_Stop(); break;

    case 'Y': isAutoMode = true; matrix_display(followmode); follow(); break;
    case 'U': isAutoMode = true; matrix_display(obstAvoid); avoid(); break;
    case 'X': isAutoMode = true; matrix_display(light); light_track(); break;
  }

  // After 5 seconds of being stopped, show start sign again
  if (isStopped && (millis() - lastStopTime >= 1000)) {
    matrix_display(start01);
    isStopped = false;
  }
}

void avoid() {
  isStopped = false;
  flag = 0;
  while (flag == 0) {
    random2 = random(1, 100);
    a = checkdistance();

    if (a < 20) {
      Tank_Stop();
      delay(200);

      procedure(160);
      a1 = 0;
      for (int j = 0; j < 5; j++) a1 += checkdistance();
      a1 /= 5;

      delay(200);
      procedure(20);
      a2 = 0;
      for (int k = 0; k < 5; k++) a2 += checkdistance();
      a2 /= 5;

      procedure(90);

      if (a1 > a2) {
        Tank_left();
        delay(500);
      } else {
        Tank_right();
        delay(500);
      }
      Tank_front();
    } else {
      Tank_front();
    }

    if (Serial.available()) {
      bluetooth_val = Serial.read();
      if (bluetooth_val == 'Z') flag = 1;
    }
  }
```

RC_Tank

```
    if (Serial.available()) {
      bluetooth_val = Serial.read();
      if (bluetooth_val == 'Z') flag = 1;
    }
  }
}

void follow() {
  flag = 0;
  while (flag == 0) {
    distance = checkdistance();

    if (distance >= 10 && distance <= 20) {
      Tank_front();
    } else {
      Tank_Stop();
      isStopped = false;
    }

    if (Serial.available()) {
      bluetooth_val = Serial.read();
      if (bluetooth_val == 'Z') {
        flag = 1;
      }
    }
  }
}

float checkdistance() {
  long sum = 0;
  for (int i = 0; i < 5; i++) {
    digitalWrite(Trig, LOW);
    delayMicroseconds(2);
    digitalWrite(Trig, HIGH);
    delayMicroseconds(10);
    digitalWrite(Trig, LOW);
    sum += pulseIn(Echo, HIGH);
    delay(10);
  }
  return (sum / 5.0) / 58.0;
}

void procedure(int myangle) {
  int pulsewidth;
  for (int i = 0; i <= 50; i++) {
    pulsewidth = myangle * 11 + 500;
    digitalWrite(servoPin, HIGH);
    delayMicroseconds(pulsewidth);
    digitalWrite(servoPin, LOW);
    delay(20 - pulsewidth / 1000);
  }
}

void light_track() {
  flag = 0;
  while (flag == 0) {
    left_light = analogRead(light_L_Pin);
    right_light = analogRead(light_R_Pin);

    if (left_light > 650 && right_light > 650) Tank_front();
    else if (left_light > 650) Tank_left();
    else if (right_light > 650) Tank_right();
    else Tank_Stop();
    isStopped = false;
```

RC_Tank

```
    if (left_light > 650 && right_light > 650) Tank_front();
    else if (left_light > 650) Tank_left();
    else if (right_light > 650) Tank_right();
    else Tank_Stop();
    isStopped = false;

    if (Serial.available()) {
      bluetooth_val = Serial.read();
      if (bluetooth_val == 'Z') flag = 1;
    }
  }
}

void matrix_display(unsigned char matrix_value[]) {
  IIC_start();
  IIC_send(0xc0);
  for (int i = 0; i < 16; i++) IIC_send(matrix_value[i]);
  IIC_end();
  IIC_start();
  IIC_send(0x8A);
  IIC_end();
}

void IIC_start() {
  digitalWrite(SCL_Pin, HIGH); delayMicroseconds(3);
  digitalWrite(SDA_Pin, HIGH); delayMicroseconds(3);
  digitalWrite(SDA_Pin, LOW); delayMicroseconds(3);
}

void IIC_send(unsigned char send_data) {
  for (char i = 0; i < 8; i++) {
    digitalWrite(SCL_Pin, LOW); delayMicroseconds(3);
    digitalWrite(SDA_Pin, (send_data & 0x01) ? HIGH : LOW);
    delayMicroseconds(3);
    digitalWrite(SCL_Pin, HIGH); delayMicroseconds(3);
    send_data >>= 1;
  }
}

void IIC_end() {
  digitalWrite(SCL_Pin, LOW); delayMicroseconds(3);
  digitalWrite(SDA_Pin, LOW); delayMicroseconds(3);
  digitalWrite(SCL_Pin, HIGH); delayMicroseconds(3);
  digitalWrite(SDA_Pin, HIGH); delayMicroseconds(3);
}
```

```
void IIC_end() {
  digitalWrite(SCL_Pin, LOW); delayMicroseconds(3);
  digitalWrite(SDA_Pin, LOW); delayMicroseconds(3);
  digitalWrite(SCL_Pin, HIGH); delayMicroseconds(3);
  digitalWrite(SDA_Pin, HIGH); delayMicroseconds(3);
}

void Tank_front() {
  digitalWrite(MR_Ctrl, LOW); analogWrite(MR_PWM, 200);
  digitalWrite(ML_Ctrl, LOW); analogWrite(ML_PWM, 200);
}

void Tank_back() {
  digitalWrite(MR_Ctrl, HIGH); analogWrite(MR_PWM, 200);
  digitalWrite(ML_Ctrl, HIGH); analogWrite(ML_PWM, 200);
}

void Tank_left() {
  digitalWrite(MR_Ctrl, LOW); analogWrite(MR_PWM, 255);
  digitalWrite(ML_Ctrl, HIGH); analogWrite(ML_PWM, 255);
}

void Tank_right() {
  digitalWrite(MR_Ctrl, HIGH); analogWrite(MR_PWM, 255);
  digitalWrite(ML_Ctrl, LOW); analogWrite(ML_PWM, 255);
}

void Tank_Stop() {
  digitalWrite(MR_Ctrl, LOW); analogWrite(MR_PWM, 0);
  digitalWrite(ML_Ctrl, LOW); analogWrite(ML_PWM, 0);

  if (!isAutoMode) {
    matrix_display(STOP01);
    lastStopTime = millis();
    isStopped = true;
  } else {

  }
}
```

Results and Discussion

The robot tank performed flawlessly in automated as well as manual modes, validating the quality and worth of its included constituents and control logic. Key findings were recorded in the responsiveness of the tank, perception of the surroundings, and dependability in different application cases.

Manual Control

Manual control through Bluetooth was smooth and with near zero latency. The Bluetooth module HS-06 was firmly connected with the Android application throughout the test. Directional input ('F', 'B', 'L', 'R', 'S') were read in real-time by the Arduino Uno R3, and the corresponding motor actions and corresponding LED matrix animations were performed. LED display feedback was provided for enhanced usability, such as the clear indication of movement direction.

The manual running problem was that it would oversteer when making sharp turns at high PWM inputs. It was fixed by modifying the code such that it turns the motors slightly slower during turns for the direction change to be smooth.

Obstacle Avoidance Mode

In the obstacle avoidance mode, the tank relied on real-time data from the ultrasonic sensor for autonomous movement. When an object was detected at 20 cm, the robot would stop correctly, sweep its ultrasonic, and make intelligent direction decisions based on room available. It eliminated incorrect distance numbers created from sharp reflections or sloped surfaces through the averaging of multiple numbers as it swept its sensors.

This mode functioned well in dynamic environments. It functioned, however, poorly in case of narrow passages or environments with very reflective surfaces, leading to the occasional error in calculations. Despite that, the tank remained able, in general, to select collision-free paths.

Object Following Mode

The tracking algorithm made the tank track and follow the moving object—a hand or any other object—over some distance (10–20 cm). It was very stable if the moving object traversed in continuous and gradual motion. When the object moved too rapidly or suddenly left the detection area, it would pause, as per its programming, until being regained.

This mode depended substantially on ultrasonic accuracy. It might have been enhanced if it used supplementary side-looking sensors in order not to have blind spots, but it wasn't part of the initial implementation.

Light Tracking Mode

The two photoresistor light-tracker functioned well in high light conditions. The robot rotated in the direction of the high light on the tank side and moved ahead in the case of balanced

high intensity for the two sensors. In low light conditions, the action was not very consistent, prone to the stop state since there wasn't much contrast in the sensory inputs.

Sensor sensitivity would be permissible, albeit supplemented under poor light through calibration or advanced analogue filtering. In the future, an improvement would be the use of higher accuracy digital ambient light sensors.

Overall System Performance

The use of modular functions in handling the movement and the sensors made the code base extremely debuggable and extensible. The combination of Bluetooth control and the display output with LEDs made the robot extremely interactive and versatile. All the three modes of autonomous running worked as anticipated, with no clash between the sensor inputs.

Overall, the robot tank was an excellent multi-mode vehicle that was capable of conducting both user-initiated and sensor-based behaviours. It was capable of being equipped for advanced object manipulation and route following tasks with the enhanced calibration and provision of extra sensors.

References

- Electronics Tutorials, 2025. *HC-SR04 Ultrasonic Distance Sensor Tutorial*. Available at: https://www.electronics-tutorials.ws/io/io_5.html (Accessed: 12 May 2025).
- **Texas Instruments, 2025.** *CC2564 Bluetooth® 4.0 with Enhanced Data Rate (EDR), Low Energy (LE), and ANT*. Available at: https://www.ti.com/lit/ds/symlink/cc2564.pdf (Accessed: 17 May 2025).
- **Handson Technology, 2025.** *HC-SR04 Ultrasonic Sensor Module User Manual*. Available at: https://www.handsontec.com/dataspecs/HC-SR04-Ultrasonic.pdf (Accessed: 17 May 2025).
- **Guangzhou HC Information Technology Co., Ltd., 2010.** *HC-06 Bluetooth Serial Module Datasheet*. Available at: https://www.olimex.com/Products/Components/RF/BLUETOOTH-SERIAL-HC-06/resources/hc06.pdf (Accessed: 17 May 2025).
- **Gordon, S., 2025.** *Dot Matrix Tool – LCD Font Generator*. Available at: https://dotmatrixtool.com/ (Accessed: 21 May 2025).